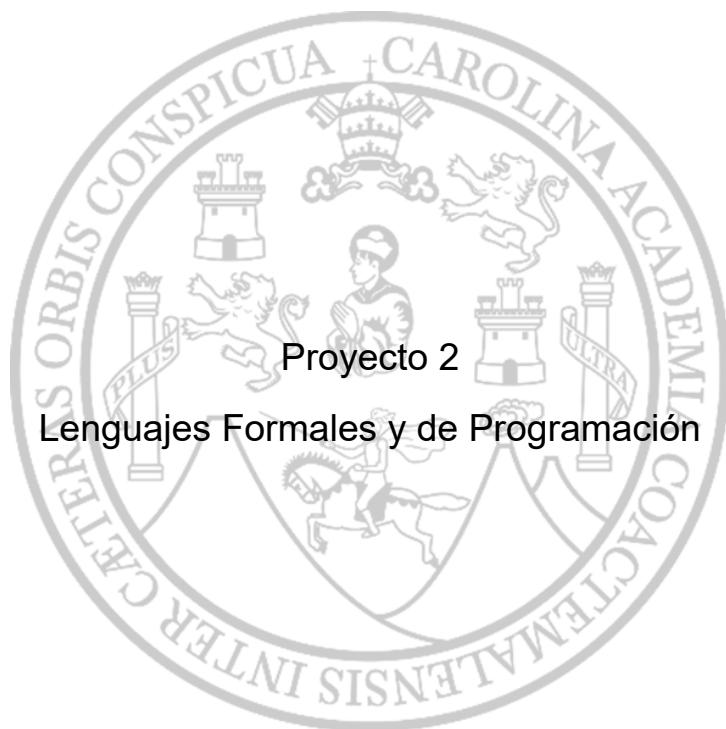


UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería

Escuela de Ciencias y Sistemas



Proyecto 2

Lenguajes Formales y de Programación

Nombre: Jose Juan Laynez Zapeta

Fecha: 01/05/2026

Programa TaskScript:

El programa realizado busca la lectura de archivos .Task. Al ingresar un archivo en el frame principal, el usuario debe ser capaz de recibir reportes basados en el archivo ingresado. Esto dependerá en gran manera del análisis realizado, ya que si el análisis sintáctico muestra errores, no se podrá generar ningún reporte. Se realizará un análisis léxico previo al sintáctico para el reconocimiento de tokens.

Arquitectura del Sistema:

Dentro del programa TaskScript, se manejan clases para el análisis léxico y sintáctico, estos a su vez, comparten la misma clase ErrorManager, la cual, sirve para almacenar errores.

Estos son miembros de la clase MainFrame, la cual, funciona como panel principal del programa, y la cual, llama a los métodos de las clases anteriores por medio de botones accionados por el usuario.

Cabe resaltar que se usó wxWidgets para el desarrollo del programa, así que se usaron algunas funciones y propiedades de esta para la elaboración del programa.

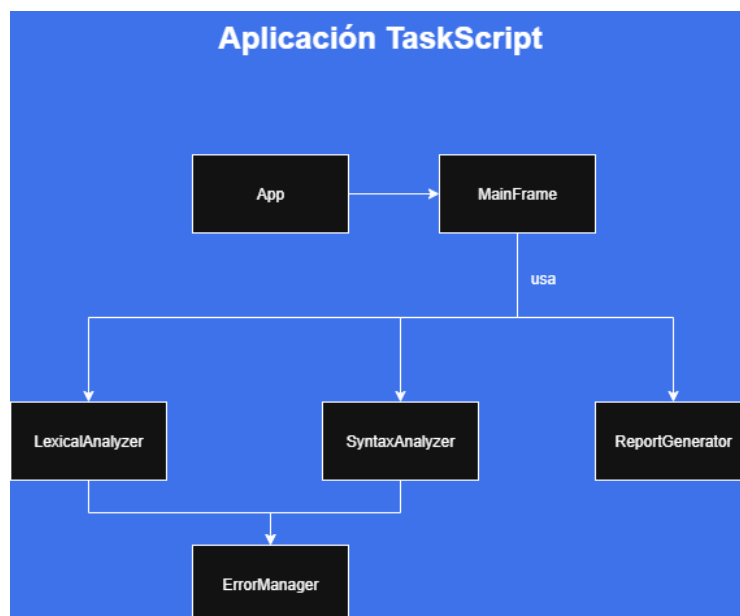
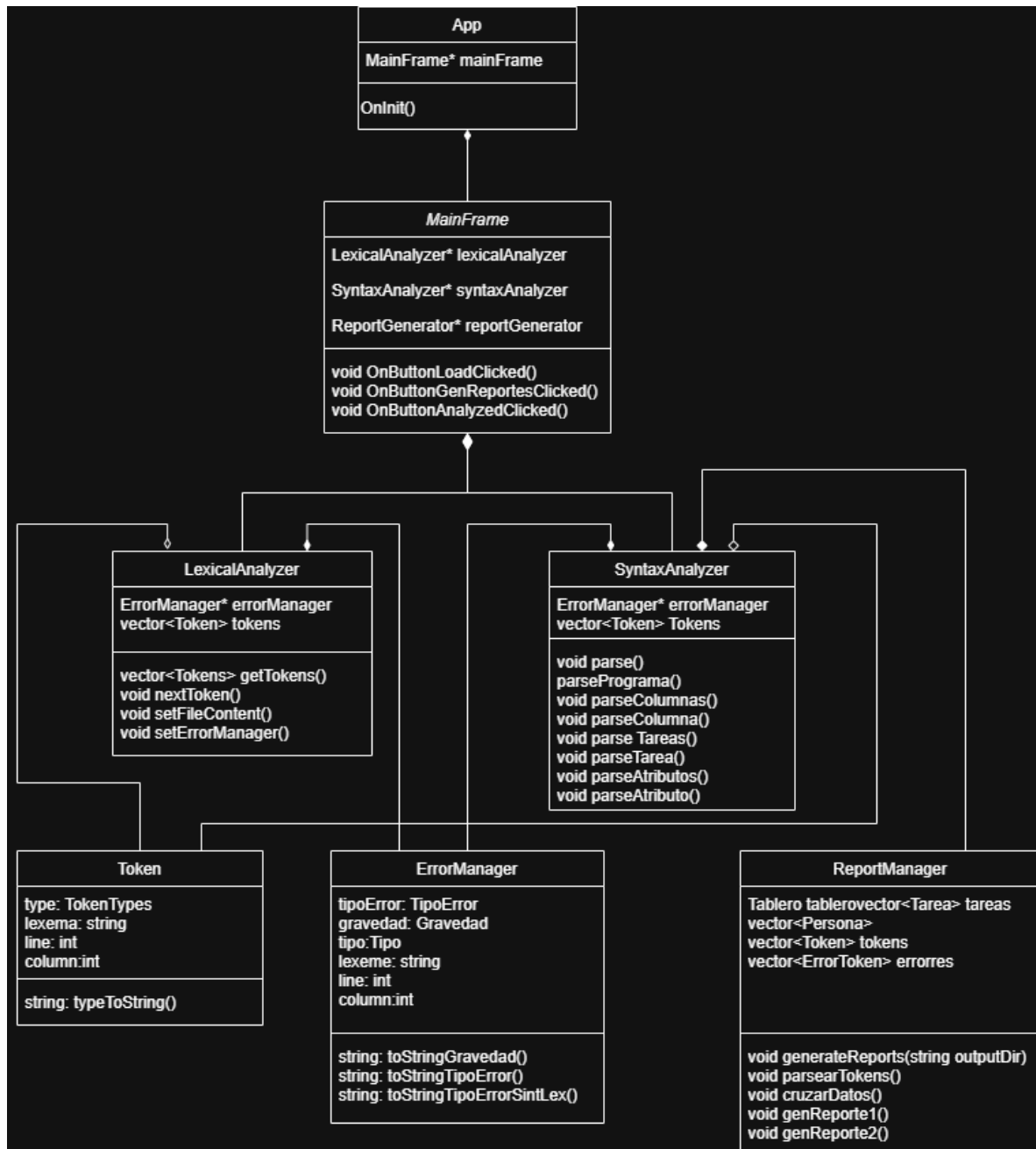
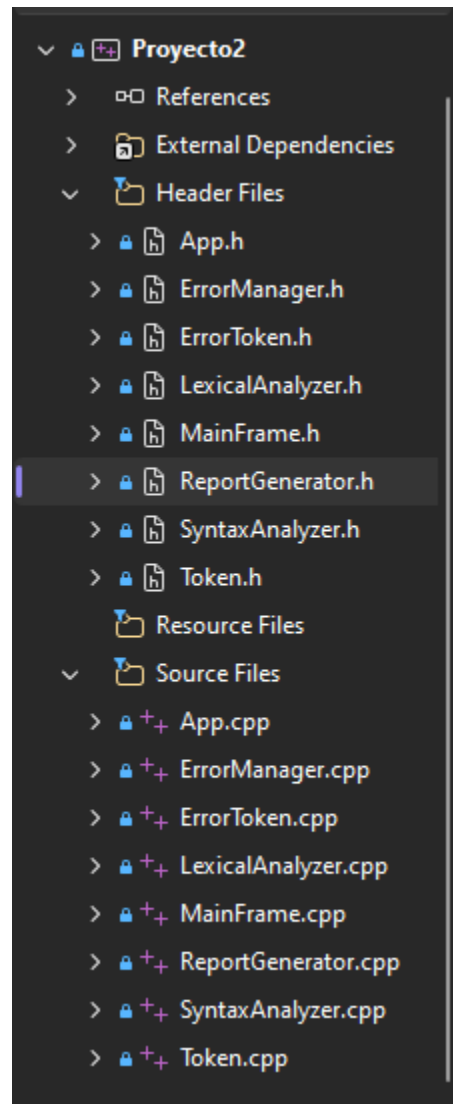
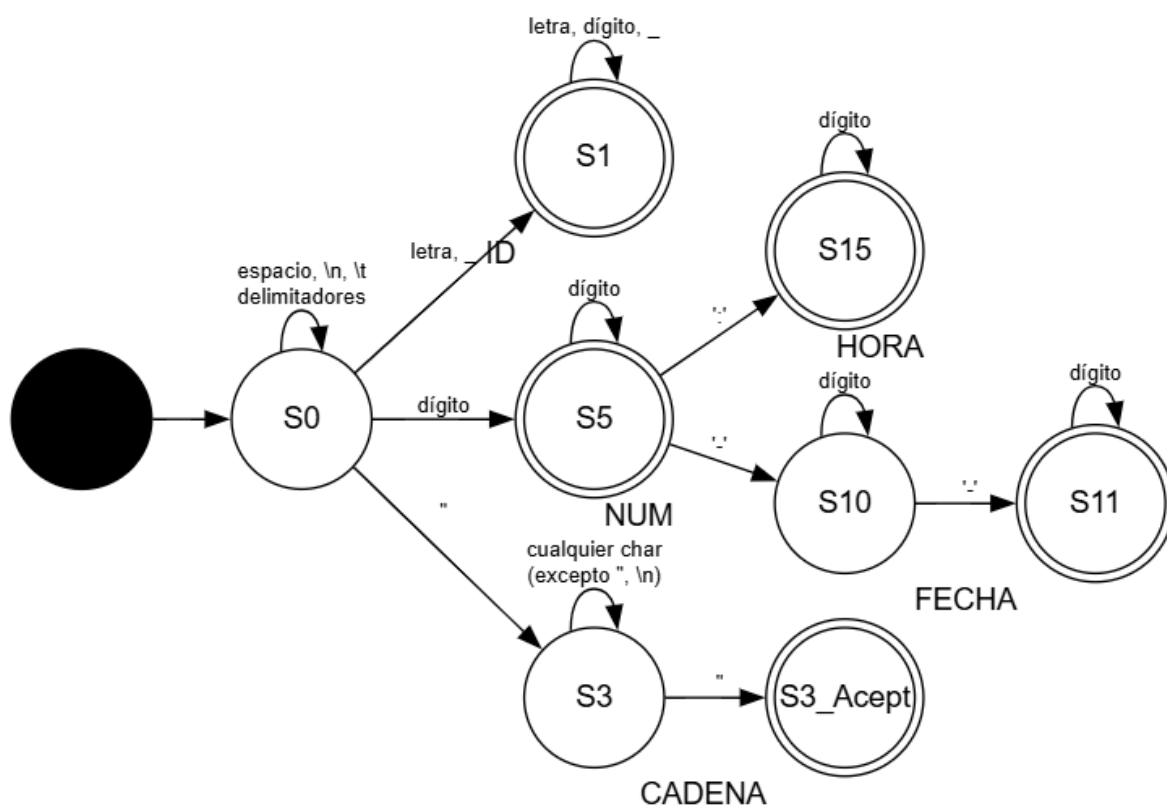


Diagrama de clases:



Clases dentro de Visual Studio:





Gramática Libre de Contexto:

<programa> ::= TABLERO CADENA "{" <columnas> "}" ";"

<columnas> ::= <columna> <columnas>

| <columna>

<columna> ::= COLUMNA CADENA "{" <tareas> "}" ";"

| COLUMNA CADENA "{" "}" ";"

<tareas> ::= <tarea> "," <tareas>

| <tarea> ";"

| <tarea>

<tarea> ::= tarea ":" CADENA "[" <atributos> "]"

<atributos> ::= <atributo> "," <atributos>

| <atributo> ";"

| <atributo>

<atributo> ::= prioridad ":" <prioridad>

| responsable ":" CADENA

| fecha_limite ":" FECHA

<prioridad> ::= ALTA | MEDIA | BAJA

Estructura SyntaxAnalyzer (El parser):

El parser descendente recursivo usado se basa principalmente en el uso de Tokens, los cuales, son obtenidos con el análisis léxico desde LexicalAnalyzer().

LexicalAnalyzer usa el mismo ErrorManager() que, para añadir el error junto con los demás. Este parser usa las siguientes clases.

```
1  #pragma once
2  #include <vector>
3  #include "Token.h"
4  #include "ErrorManager.h"
5  #include <stdexcept>
6
7  class SyntaxError : public std::runtime_error {
8  public:
9      SyntaxError(const std::string& msg) : std::runtime_error(msg) {}
10     }
11 };
12
13
14 class SyntaxAnalyzer
15 {
16
17 private:
18     std::vector<Token> tokens;
19     int pos;
20     ErrorManager* errorManager;
21
22     Token peek();
23     Token consume();
24     bool match(TokenTypes type);
25     bool matchLexeme(const std::string& lexeme);
26     Token expect(TokenTypes type);
27     Token expectLexeme(const std::string& lexeme);
28     bool isAtEnd();
29
30     void parsePrograma();
31     void parseColumnas();
32     void parseColumna();
33
34     void parseTablero();
35     void parseTareas();
36     void parseTarea();
37     void parseAtributos();
38     void parseAtributo();
39
40 public:
41     SyntaxAnalyzer(std::vector<Token> t, ErrorManager* eM);
42     void parse();
43 };
44
45
```

El parser funciona en base al token actual, en base a ese, sin mirar atrás, busca qué debería seguirle, según la GLC planeada.

Peek() se encarga de revisar el token actual, retornando ese Token para su revisión o comparación. Funciona en conjunto con isAtEnd(), que básicamente indica si ya se llegó al final o no.

```

bool SyntaxAnalyzer::isAtEnd() {
    return pos >= (int) tokens.size();
};

Token SyntaxAnalyzer::peek() {
    if (!isAtEnd()) {
        return tokens[pos];
    }

    return Token(TokenTypes::UNDEFINED, "", -1, -1);
};

```

Consume() básicamente retorna el Token actual como Peek(), pero también avanza a la siguiente posición, lo cual Peek() no hace. Es útil para comparaciones y avanzar al siguiente Token sin directamente declararlo como pos++ en conjunto con Peek().

```

Token SyntaxAnalyzer::consume() {
    if (!isAtEnd()) {
        return tokens[pos++];
    }
    else {
        return Token(TokenTypes::UNDEFINED, "", -1, -1);
    }
};

```

Match() indicará si el tipo del token donde estamos (por medio de peek()) es el mismo al tipo el cual se ingresa como parámetro de la función. Será muy útil a la hora de construir otras secciones del parser. MatchLexeme() se usa para lo mismo, pero más que todo para comprar si el lexema es el mismo al del que se requiere. Se agregó en dado caso fuera a usarse (aunque no se usó).

```

bool SyntaxAnalyzer::match(TokenTypes tType) {
    return (!isAtEnd() && (peek().type == tType));
}

bool SyntaxAnalyzer::matchLexeme(const std::string& lexeme) {
    return (!isAtEnd() && (peek().lexeme == lexeme));
};

```

Finalmente, las funciones más usadas del parser son expect() y expectToken(). Estas devuelven un Token, sin embargo, cumplen la función vital del parser, de ejecutar las funciones anteriores. Dependiendo si el Token o Lexema es el esperado (con match()) y

matchLexeme()), usa consume() para ir al siguiente y Token, o en caso falle, tira una excepción y detiene el programa y añade el error a la lista.

```
40 Token SyntaxAnalyzer::expect(TokenTypes type){
41     if (match(type)) {
42         return consume();
43     }
44
45     Token malo = peek();
46     errorManager->addError(malo.lexema, malo.line, malo.column, TipoError::UNEXPECTED_TOKEN, Gravedad::CRITICO, Tipo::SINTACTICO);
47
48     throw SyntaxError("Error Sintáctico " + std::to_string(malo.line));
49 };
50
51 Token SyntaxAnalyzer::expectLexeme(const std::string& lexeme) {
52     if (matchLexeme(lexeme)) {
53         return consume();
54     }
55
56     Token malo = peek();
57     errorManager->addError(malo.lexema, malo.line, malo.column, TipoError::UNEXPECTED_TOKEN, Gravedad::CRITICO, Tipo::SINTACTICO);
58     return Token(TokenTypes::UNDEFINED, malo.lexema, malo.line, malo.column);
59 };
```

Se ejecuta al parseo desde afuera con la función parse(), y el error se atrapa con un try catch.

```
62 void SyntaxAnalyzer::parse() {
63     try{
64         parsePrograma();
65     }
66     catch (const SyntaxError&) {
67     }
68 };
```

Con todo esto explicado, se puede revisar mas a fondo el parser, el cuál, está basado en todo lo anterior.

Como se puede ver en la captura anterior, parse() llama de primero a parsePrograma(). Lo que hace parse programa es el comienzo del parseo. En base a expect(), revisa si los Tokens siguen la estructura de Tipos de Token, en medio de esto, se parsean las columnas.

```
72 // <Programa> ::= TABLERO CADENA "{" <columnas> "}" ";"
73
74
75 void SyntaxAnalyzer::parsePrograma() {
76     expect(TokenTypes::RESERVADA_MAIN);
77     expect(TokenTypes::LIT_CADENA);
78     expect(TokenTypes::LLAVE_ABRE);
79     parseColumnas();
80     expect(TokenTypes::LLAVE_CIERRA);
81     expect(TokenTypes::PUNTO_Y_COMA);
82 }
```

Se espera que la tabla tenga por lo menos una columna, pero si tiene mas, se encapsula dentro de un ciclo while parseColumnas() para que siga con el parseando el contenido de las siguientes columnas.

```
84 //<Columnas> ::= <columna> <columnas> | <Columna>
85
86 void SyntaxAnalyzer::parseColumnas() {
87     parseColumna();
88     while (match(TokenTypes::RESERVADA_SUB)) {
89         parseColumna();
90     }
91 }
```

Ahora bien, parseColumnas() parsea los contenidos de una columna, dentro de ella, se llama a parseTareas() para parsear los contenidos de la o las tareas.

```
93 // <columna> ::= COLUMNA CADENA "{" <tareas> "}" ";"
94
95
96 void SyntaxAnalyzer::parseColumna() {
97     expect(TokenTypes::RESERVADA_SUB);
98     expect(TokenTypes::LIT_CADENA);
99     expect(TokenTypes::LLAVE_ABRE);
100     if (match(TokenTypes::RESERVADA_ELEMENTO)) {
101         parseTareas();
102     }
103     expect(TokenTypes::LLAVE_CIERRA);
104     expect(TokenTypes::PUNTO_Y_COMA);
105 }
```

El parseo de las tareas llama al parseo de una tarea singular, similar a como se hizo en el de columnas. Se revisa si hay separación con comas y si hay, se parsea la tarea (conocido como ELEMENTO). Se espera por lo menos uno, pero si hay mas, se verifica con un match(), sino hay mas, continúa.

```
107 //<tareas> ::= <tarea> "," <tareas> | <tareas>
108
109 void SyntaxAnalyzer::parseTareas() {
110     parseTarea();
111     while (match(TokenTypes::COMA)) {
112         consume();
113         if (!match(TokenTypes::RESERVADA_ELEMENTO)) {
114             break;
115         }
116         parseTarea();
117     }
118 }
119
120
121 }
```

Se revisan las tareas, con `parseTarea()`, siempre con `expect()` se revisan si los tipos de Tokens que siguen en el vector coinciden con los de los Tokens que siguen según la GLC.

```
123 //<tarea>::=tarea ":" CADENA "[" <atributo> "]"
124
125 void SyntaxAnalyzer::parseTarea() {
126     expect(TokenTypes::RESERVADA_ELEMENTO);
127     expect(TokenTypes::DOS_PUNTOS);
128     expect(TokenTypes::LIT_CADENA);
129     expect(TokenTypes::CORCHETE_ABRE);
130     parseAtributos();
131     expect(TokenTypes::CORCHETE_CIERRA);
132 };
```

Finalmente, se parsean los atributos de las Tareas con `parseAtributos()`, el cual, a su vez, llama a `parseAtributo()`. El GLC está diseñado para que solo se acepten de tres tipos, `fecha_limite`, `responsable`, y `prioridad`.

```
136 void SyntaxAnalyzer::parseAtributos() {
137     parseAtributo();
138     while (match(TokenTypes::COMA)) {
139         consume();
140         if (!match(TokenTypes::RESERVADA_ATRIBUTO)) {
141             break;
142         }
143         parseAtributo();
144     }
145 };
```

```
// <atributo>::= prioridad ":" <prioridad> | responsable ":" CADENA | fecha_limite ":" FECHA
void SyntaxAnalyzer::parseAtributo() {
    Token attrib = expect(TokenTypes::RESERVADA_ATRIBUTO);
    expect(TokenTypes::DOS_PUNTOS);

    if (attrib.lexema=="prioridad") {
        expect(TokenTypes::NIVEL_PRIORIDAD);
    }
    else if (attrib.lexema == "responsable") {
        expect(TokenTypes::LIT_CADENA);
    }
    else if (attrib.lexema == "fecha_limite") {
        expect(TokenTypes::LIT_FECHA);
    }
    else {
        if (!isAtEnd() && !match(TokenTypes::COMA) && !match(TokenTypes::CORCHETE_CIERRA)) {
            consume();
        }
    }
};
```

Decisiones en el código y su estructura:

ErrorManager compartido: tanto el léxico como el sintáctico reportan al mismo objeto, esto hace que sea más simple la recolección de errores y que se pueda ver todos en una sola lista, de ser necesario.

Parada al primer error sintáctico: dado que la gramática es secuencial y un error en la estructura del tablero haría que todos los datos extraídos fueran incorrectos, es más seguro detener el análisis que intentar recuperarse y generar reportes con datos parciales o erróneos.

wxWidgets para la GUI: es nativa, multiplataformas, sin dependencias pesadas. Se integra bien con la carga de archivos y para presentar los datos.